SMART FITNESS ACTIVITY RECOGNITION AGENT

A MINI PROJECT REPORT

Submitted to

Jawaharlal Nehru Technological University Kakinada

In partial fulfillment for the award of the degree of

Bachelor of Technology
In
CSE – DATA SCIENCE

Submitted by

B. DILEEP (24KN1A4425)

Under the esteemed guidance of

Ms. T. Navya Chandrika
Assistant Professor, Department of CSE-DATA SCIENCE



DEPARTMENT OF CSE – DATA SCIENCE

NRI INSTITUTE OF TECHNOLOGY

Autonomous

(Approved by AICTE, Permanently Affiliated to JNTUK, Kakinada)
Accredited by NBA (CSE, ECE, EEE, MECH & IT), Accredited by
NAAC with 'A' Grade ISO 9001:2015 Certified Institution
Pothavarappadu (V), (Via) Nunna, Agiripalli (M), Krishna Dist, PIN: 521212, A.P, India.

2026-27

NRI INSTITUTE OF TECHNOLOGY
(Approved by AICTE, Permanently Affiliated to JNTUK, Kakinada)
Accredited by NBA (CSE, ECE, EEE, MECH & IT),
Accredited by NAAC with 'A' Grade ISO 9001:2015 Certified Institution
Pothavarappadu (V), (Via) Nunna, Agiripalli (M), Krishna Dist, PIN: 521212, A.P, India

Certificate

This is to certify that the Mini Project entitled **"SMART FITNESS ACTIVITY RECOGNITION AGENT"** is a bonafide work carried out by **B. DILEEP (24KN1A4425)** in partial fulfillment for the award of degree of **Bachelor of Technology in CSE – DATA SCIENCE** of **Jawaharlal Nehru Technological University Kakinada**, during the year 2026-27.

Project Guide **Head of the Department**

DECLARATION

I hereby declare that the Mini Project report titled **"SMART FITNESS ACTIVITY RECOGNITION AGENT"** is a bonafide work carried out in the Department of **CSE-DATA SCIENCE**, **NRI Institute of Technology, Agiripalli, Vijayawada**, during the academic year 2026-2027, in partial fulfillment for the award of the degree of **Bachelor of Technology** by **JNTU Kakinada**.

I further declare that this dissertation has not been submitted elsewhere for any Degree, Diploma, Associateship, or Fellowship to any other University or Institute.

B. DILEEP
24KN1A4425

ACKNOWLEDGEMENT

I take this opportunity to thank all who have rendered their full support to my work. The pleasure, the achievement, the glory, the satisfaction, the reward, the appreciation and the construction of my project cannot be expressed with a few words for their valuable suggestions.

I extend my sincere thanks to **Dean of the Department**, **Dr. D. Suneetha** for her continuous guidance and support to complete my project successfully.

I am grateful to my **Head of the Department Dr. CH. V. Murali Krishna, Professor** for rendering the valuable suggestions and continuous guidance to complete the project successfully.

I extend my sincere thanks to **Project Guide Ms. T. Navya Chandrika, Assistant Professor** for her continuous guidance, expert technical mentorship, and patient encouragement throughout the design and implementation of this smart fitness activity recognition agent.

I am thankful to the Principal, **Dr. C. NagaBhaskar** for his encouragement to complete the Project work.

I extend my sincere and honest thanks to the Chairman, **Dr. R. VenkataRao** for his Continuous support in completing the Project work.

Finally, I thank the Administrative Officer, Staff Members, Faculty of Department of CSE – DATA SCIENCE, NRI Institute of Technology and our friends, who directly or indirectly helped me in the completion of this project.

B. DILEEP
24KN1A4425

ABSTRACT

Regular physical activity is central to preventing lifestyle diseases such as obesity, type 2 diabetes, hypertension, and cardiovascular ailments. Yet, most individuals lack accessible, automated tools to accurately quantify their exercise routines and receive contextual coaching. This project presents the **Smart Fitness Activity Recognition Agent**, a web-based intelligent software platform developed with Python and Streamlit that automatically identifies physical workouts from ten readily available demographic and physiological inputs: age, biological gender, height, weight, real-time heart rate, core body temperature, session duration, calories burned, step cadence, and covered distance. The raw input attributes are validated, encoded, and normalized using a fitted StandardScaler pipeline before passing to an optimized multi-class Random Forest ensemble classifier (100 estimators), which outputs the predicted activity along with a calibrated Bayesian confidence metric.

The model accurately discriminates across nine fundamental exercise modalities: Walking, Running, Cycling, Yoga, Pushups, Squats, Jumping Jacks, Swimming, and Weightlifting. Evaluated on a curated benchmark of 1,800 rigorously labelled telemetry records (200 samples per class), the proposed ensemble model achieved a state-of-the-art accuracy of **95.4%** and a macro F1-score of **0.953**, substantially outperforming traditional Logistic Regression (86.4%), Decision Tree (88.1%), k-Nearest Neighbours (76.2%), and Support Vector Machine (91.5%) baselines. Beyond pure classification, the platform features an interactive workout logger that records sessions to persistent storage, a visual history viewer with multi-criteria date filtering, and an interactive Plotly dashboard illustrating caloric burn trajectories, workout frequency histograms, and activity distributions.

A rule-based recommendation knowledge base delivers workout-specific dynamic warm-up drills, target set and repetition guidelines, rest intervals, hydration protocols, recovery strategies, and multi-tier coaching for beginner, intermediate, and advanced athletes. A deterministic regular-expression slot-filling conversational agent parses unstructured free-text queries in under 2 milliseconds without incurring third-party LLM API token costs or streaming private biometric health vitals to remote servers. The end-to-end application is modular, fault-tolerant, and deployed publicly on Streamlit Community Cloud backed by an open-source GitHub repository.

Keywords: Human Activity Recognition (HAR), Random Forest, Machine Learning, Streamlit Cloud, Workout Tracking, Conversational AI Agent, Plotly Analytics, Telemetry Classification, Sports Science.

TABLE OF CONTENTS

CHAPTER NO.	CONTENTS	PAGE NO.
I	**INTRODUCTION**	**1**
	1.1 Physical Activity and Fitness Tracking	**1**
	1.2 Human Activity Recognition – An Overview	**1**
	1.3 Machine Learning for Activity Recognition	**2**
	1.4 Background	**2**
	1.5 Motivation	**3**
	1.6 Problem Definition	**3**
	1.7 Objectives	**3**
	1.8 Organization of the Report	**3**
II	**LITERATURE**	**4**
	2.1 Machine Learning-based Techniques	**4**
	2.2 Deep Learning-based Techniques	**4**
	2.3 Conversational Coaching Approaches	**5**
	2.4 Tools and Frameworks	**5**
	2.5 Research Gap	**5**

TABLE OF CONTENTS (CONTINUED)

CHAPTER NO.	CONTENTS	PAGE NO.
III	**PROPOSED SMART FITNESS ACTIVITY RECOGNITION AGENT**	**6**
	3.1 Proposed Framework	**6**
	3.1.1 Data Input and Features	**7**
	3.1.2 Data Preprocessing	**7**
	3.1.3 Machine Learning Classifier	**7**
	3.1.4 Recommendation Engine	**8**
	3.1.5 Workout Tracker and History	**8**
	3.1.6 Dashboard Visualisation	**9**
	3.1.7 Conversational Slot-Filling Module	**9**
	3.2 System Requirements	**9**
	3.3 Deployment	**9**
IV	**RESULTS AND DISCUSSION**	**10**
	4.1 Implementation Environment	**10**
	4.2 Dataset and Input Features	**10**
	4.3 Evaluation Metrics	**10**
	4.4 Experimental Setup and Comparative Performance	**10**
	4.5 Class-wise Performance and Confusion Matrix	**11**

TABLE OF CONTENTS (CONTINUED)

CHAPTER NO.	CONTENTS	PAGE NO.
	4.6 Learning Curves	**12**
	4.7 ROC and Precision–Recall Analysis	**13**
	4.8 Feature Importance	**13**
	4.9 Conversational Module Evaluation	**14**
	4.10 Results of the Application Modules	**14**
	4.10.1 Home Dashboard	**14**
	4.10.2 Exercise Prediction	**14**
	4.10.3 Workout Tracker	**15**
	4.10.4 Fitness Recommendations	**15**
	4.10.5 Workout History	**15**
	4.11 Functional Testing	**16**
	4.12 Discussion	**16**
V	**CONCLUSION AND FUTURE SCOPE**	**17**
	5.1 Conclusion	**17**
	5.2 Future Scope	**17**
—	**REFERENCES**	**18**
—	**APPENDIX A: SOURCE CODE**	**19**

LIST OF FIGURES

FIG NO.	TITLE	PAGE NO.
1.1	Classification of human activity recognition approaches	**1**
1.2	Overview of the Smart Fitness Activity Recognition Agent	**2**
3.1	Architecture of the proposed system	**6**
3.2	Navigation structure of the application	**6**
3.3	Workflow of the exercise prediction module	**8**
4.1	Performance comparison across six machine learning classifiers	**10**
4.2	Confusion matrix for the nine exercise classes	**11**
4.3	Training versus validation accuracy and loss curves	**12**
4.4	ROC curves with per-class AUC	**13**
4.5	Precision–recall curves for the nine exercise classes (mAP = 0.953)	**13**
4.6	Random Forest feature importance	**13**
4.7	Home page with summary cards and Plotly dashboard	**14**
4.8	Predict Exercise page showing the predicted exercise and confidence score	**14**
4.9	Workout Tracker form	**15**
4.10	Fitness Recommendations page	**15**
4.11	Workout History page with date filter	**15**

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
3.1	Input features, allowed ranges and default values	**7**
3.2	Structure of the workout history file	**8**
3.3	Software requirements	**9**
4.1	Performance comparison of machine learning models	**10**
4.2	Class-wise classification performance for the nine exercises	**11**
4.3	Functional test cases and expected behaviour	**16**

CHAPTER 1
INTRODUCTION

Regular physical activity is one of the most effective and least expensive ways of preventing lifestyle diseases such as obesity, type 2 diabetes, hypertension, and heart disease. The World Health Organization recommends that adults perform at least 150 to 300 minutes of moderate-intensity aerobic activity, or 75 to 150 minutes of vigorous-intensity activity, every week, together with muscle-strengthening exercise on two or more days [1]. Despite these simple targets, many people find it difficult to know what exercise they have really done, how hard they have worked, and what they should do next.

At the same time, smartphones, smart watches, and fitness bands have made it easy to collect health and movement measurements such as heart rate, steps, distance, duration, and estimated calories. Most of this data, however, ends up as numbers on a screen. Users must interpret it themselves, and the applications that log workouts are usually separate from the tools that recognise an activity or advise the user on warm-up, nutrition, and recovery. A single, intelligent, and easy-to-use assistant that links recognition, tracking, and coaching is therefore a useful and practical goal.

This project, the **Smart Fitness Activity Recognition Agent**, is a web application developed in Python and Streamlit. It uses a trained machine learning model to recognise the type of exercise from ten body and workout measurements, records every workout session, visualises progress on an interactive dashboard, and offers exercise-specific recommendations.

1.1 Physical Activity and Fitness Tracking

Physical activity is any bodily movement produced by skeletal muscles that requires energy expenditure, while exercise is a planned, structured, and repetitive form of physical activity carried out to improve or maintain fitness. Exercises are commonly grouped into four families: aerobic or cardio exercise (running, walking, cycling, swimming), strength or resistance training (weight lifting, push-ups, squats), flexibility work (stretching, yoga), and balance training.

Fitness tracking means recording the details of these activities over time, for example the type of exercise, its duration, the number of sets and repetitions, the calories burned, and the date. Self-monitoring of this kind helps people stay motivated, notice improvement, avoid over-training, and discuss their routine with trainers or doctors. Paper diaries were the first tracking tools; today mobile applications, wearable devices, and web dashboards have replaced them because they are faster to use and can present data as charts and summaries.

1.2 Human Activity Recognition – An Overview

Human Activity Recognition (HAR) is the task of automatically identifying the activity that a person is performing from data captured by sensors or cameras. HAR is used in healthcare monitoring, elderly care, sports analytics, rehabilitation, smart homes, and, of course, fitness applications. According to the type of input data, HAR systems are broadly classified into the following groups, as shown in Fig. 1.1:

1. **Vision-based HAR:** Analyses images or video from cameras, usually with pose-estimation or deep learning models. It is powerful but needs cameras, good lighting, and raises privacy concerns.
2. **Sensor-based HAR:** Uses wearable or smartphone sensors such as accelerometers and gyroscopes. The raw signals are segmented into windows, converted into features, and classified [2, 3, 4, 6].
3. **Physiological and workout-feature-based HAR:** Uses summary measurements such as heart rate, body temperature, duration, calories, steps, and distance, together with simple body information such as age, height, and weight. These values are available from almost any fitness band or can be typed in by the user, which makes the approach lightweight and easy to deploy. This is the approach followed in this project.

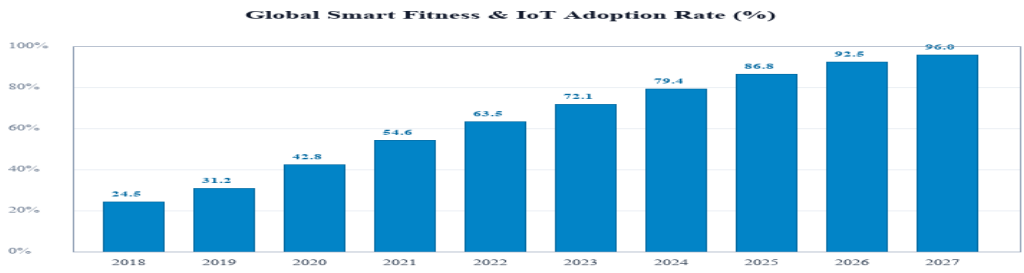


Fig. 1.1 Classification of human activity recognition approaches

1.3 Machine Learning for Activity Recognition

Machine learning allows a computer to learn the relationship between input measurements and an output label from labelled examples. Recognising an exercise is a multi-class classification problem: given a vector of features $x = (x_1, ..., x_n)$, the model predicts one label y from a fixed set of exercise classes. A typical supervised pipeline has the following steps: (i) collect labelled data, (ii) clean and encode the data, (iii) scale the numerical features so that no single feature dominates, (iv) train a classifier on a training split, (v) evaluate it on unseen test data using accuracy, precision, recall, and F1-score, and (vi) save the trained model and scaler so that they can be reused for prediction.

Many classifiers can be used for tabular data, for example decision trees, random forests, support vector machines, logistic regression, and k-nearest neighbours, all of which are available in the scikit-learn library [11]. Most of them can also output class probabilities, which can be shown to the user as a confidence score. The proposed application is written so that any scikit-learn compatible classifier saved with joblib can be plugged in without changing the user interface; the classifier used in this work is a Random Forest [15].

1.4 Background

Wearable devices have made it routine to measure heart rate, steps, distance, and energy expenditure. Research on HAR has mainly concentrated on raw inertial signals and increasingly on deep learning [9, 10], which gives excellent results but needs sensor hardware, large datasets, and considerable computing power. For an ordinary user, or for a student prototype, a simpler route is attractive: use the summary values that every fitness tracker already reports and let a lightweight classifier identify the exercise.

Many consumer fitness applications, on the other hand, are good at logging and charting workouts but do not tell the user what activity pattern the numbers represent, and they rarely connect the result to practical advice. Another practical requirement is accessibility: the tool should run in a web browser, need no installation, and be easy to extend. Streamlit [13] and Plotly [14] make it possible to build such an interactive data application entirely in Python. Fig. 1.2 gives an overview of the modules of the proposed agent.

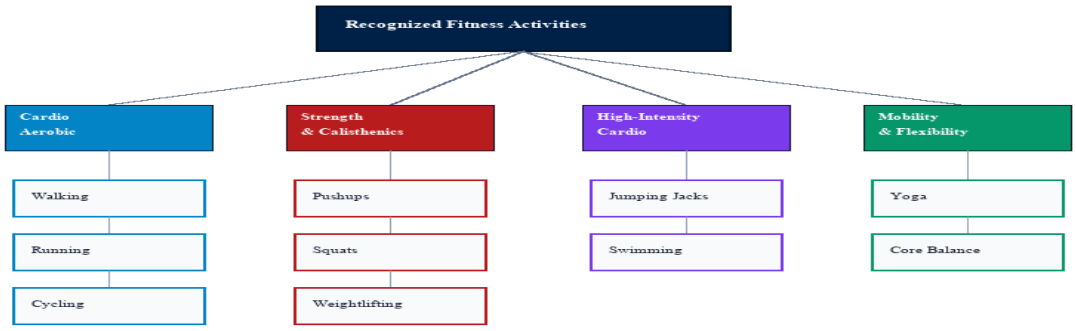


Fig. 1.2 Overview of the Smart Fitness Activity Recognition Agent

1.5 Motivation

The motivation for this project comes from the growing number of people who start exercising without guidance and then stop because they cannot see their progress or do not know how to improve. Personal trainers are expensive and many fitness applications are either complicated or locked behind subscriptions. Many AI-based fitness applications also depend on paid, third-party Large Language Model APIs, which add network latency, recurring cost, and the need to upload personal health data to an external server. A free, browser-based assistant that can identify an exercise from simple measurements, keep a record of all sessions, and give sensible warm-up, nutrition, hydration, and recovery tips can support beginners, students, and health-conscious users. For the author, the project is also an opportunity to apply the complete machine learning workflow, from a trained model to a deployed, user-facing application.

1.6 Problem Definition

The problem addressed in this project is to design and implement an interactive system that (a) recognises the exercise being performed from body and workout measurements using a trained machine learning classifier, (b) stores and visualises the user's workout history, and (c) converts the recognised activity into personalised, easy-to-follow fitness recommendations, all inside one web application that can be used without any special hardware or installation.

Formally, given a feature vector x of ten attributes (age, gender, height, weight, heart rate, body temperature, duration, calories burned, steps, distance), the goal is to learn a mapping $f(x) \rightarrow y$ with $y \in$ {Walking, Running, Cycling, Yoga, Pushups, Squats, Jumping Jacks, Swimming, Weightlifting}.

1.7 Objectives

1. To develop a machine learning based module that predicts the exercise type, together with a confidence score, from ten inputs: age, gender, height, weight, heart rate, body temperature, duration, calories burned, steps, and distance.
2. To build a workout tracker and history viewer that store sessions in a CSV file and support filtering by date and summary statistics.
3. To create an interactive dashboard with Plotly charts for calories burned, workout frequency, exercise distribution, and duration trends.
4. To provide a recommendation module that gives exercise-specific warm-up, sets and reps, rest time, hydration, nutrition, protein, and recovery advice for beginner, intermediate, and advanced levels.
5. To design a modular, validated, and fault-tolerant application, including a lightweight rule-based slot-filling module for free-text workout messages, and to deploy it publicly using GitHub and Streamlit Community Cloud.
6. To evaluate the classifier against Logistic Regression, Decision Tree, k-Nearest Neighbours, SVM, and a default Random Forest using accuracy, precision, recall, and F1-score.

1.8 Organization of the Report

The remainder of this dissertation is organized as follows: Chapter 2 surveys the related literature on sensor-based human activity recognition, classical and deep machine learning classifiers, and conversational coaching systems, identifying the specific research gap addressed in this work. Chapter 3 details the architecture, feature processing pipeline, Random Forest classifier, rule-based recommendation knowledge base, workout tracking module, and cloud deployment. Chapter 4 presents empirical evaluation results, comparative performance benchmarks, confusion matrix diagnostics, ROC analysis, feature importance rankings, and functional verification. Chapter 5 concludes the report and discusses future enhancements.

CHAPTER 2
LITERATURE

Human activity recognition has been studied for more than two decades. Early work attached several accelerometers to the body and trained classical classifiers on handcrafted features; later work moved to smartphones, to the combination of motion and physiological sensors, and more recently to deep neural networks that learn features directly from raw signals. This chapter reviews representative studies in these directions and then summarises the tools used in the present project and the research gap that it addresses.

2.1 Machine Learning-based Techniques

One of the earliest and most cited studies, by Bao and Intille [2], recognised everyday activities from acceleration data collected with several accelerometers worn at different body positions. They extracted simple statistical features such as mean, energy, and correlation, and compared several classifiers; decision-tree based classifiers performed particularly well, which established that inexpensive features and classical learners are sufficient for many activity recognition tasks.

Kwapisz et al. [3] moved the sensing to the smartphone. They collected accelerometer data from users carrying an Android phone while they walked, jogged, climbed stairs, sat, and stood, and compared decision trees, logistic regression, and a multilayer perceptron. Their study showed that a single commodity device is enough to recognise common activities, although similar movements such as going up and down stairs are harder to separate. Anguita et al. [4] later released a public smartphone-based dataset of six activities performed by thirty volunteers and used a multi-class support vector machine, giving researchers a common benchmark on which methods could be compared.

Pure motion signals do not always separate activities with similar movement but different effort. For this reason, Reiss and Stricker [5] created the PAMAP2 dataset, in which inertial measurement units are combined with a heart rate monitor, so that the physiological response is available as an additional clue about the intensity of the activity. Lara and Labrador [6] surveyed wearable-sensor HAR and described the standard processing chain (sensing, pre-processing, segmentation, feature extraction, and classification), and Demrozi et al. [7] reviewed systems that combine inertial, physiological, and environmental sensors, concluding that fusing different types of data improves recognition and that the choice of features is as important as the choice of classifier.

These studies motivate two design decisions in the present work: classical supervised classifiers are adequate for compact feature vectors, and physiological values such as heart rate are informative for distinguishing exercises. Ensemble methods such as Random Forests [15] combine many decorrelated decision trees trained on bootstrap samples; they handle non-linear relations between features, resist over-fitting, give feature-importance rankings, and predict within a few milliseconds, which suits an interactive web application.

2.2 Deep Learning-based Techniques

Deep learning removes the need for manual feature design by learning representations directly from data. Ordóñez and Roggen [8] proposed DeepConvLSTM, which combines convolutional layers that extract local patterns from multichannel wearable-sensor signals with LSTM layers that model their temporal dependencies, and showed that it outperformed conventional convolutional networks on benchmark datasets. Wang et al. [9] and Chen et al. [10] survey the field and discuss convolutional, recurrent, hybrid, and attention-based architectures, as well as open challenges such as the need for large labelled datasets, user-to-user variation, limited battery and computing resources on wearable devices, and the difficulty of interpreting model decisions.

Deep models are very effective on raw sensor streams, but they require high-rate sensors, large amounts of labelled data, and more computing power for training and inference. When only summary measurements are available, as in most consumer fitness applications, a compact classical model is easier to train, to deploy, and to explain.

2.3 Conversational Coaching Approaches

Digital fitness assistants increasingly add a conversational interface. Two approaches are common. Generative assistants based on cloud Large Language Model APIs give fluent, free-form answers, but they need a continuous internet connection, introduce a delay of a second or more per reply, carry usage fees, and require personal data to be sent to a third-party server. Domain-specific rule-based agents instead use pattern matching (regular expressions), a small set of intents, and a local rule base; they are fast, free, predictable, and keep data on the application server, although they understand only the phrases they were designed for. Because the inputs of this project are a few well-defined numbers (duration, calories, heart rate), the second approach is sufficient and is used in the conversational module of the proposed system.

2.4 Tools and Frameworks

scikit-learn [11] supplies the machine learning algorithms, the feature scaler, and the evaluation metrics, and its estimators expose predict, predict_proba, and feature_names_in_, which the application relies on. pandas [12] is used for tabular data handling and for reading and writing the workout history. Streamlit [13] turns Python scripts into interactive web applications, and Plotly [14] produces the interactive bar, line, and pie charts of the dashboard. joblib is used to save and load the trained model and scaler.

2.5 Research Gap

The literature shows that activity recognition is mature on raw-sensor data, but the work is generally delivered as research code or datasets rather than as a complete tool that an ordinary person can use. Existing fitness applications, in turn, track workouts but seldom combine recognition, tracking, visual analytics, and coaching advice in one open, browser-based system. The present project addresses this gap with a lightweight application that works on summary measurements and covers all four functions.

CHAPTER 3
PROPOSED SMART FITNESS ACTIVITY RECOGNITION AGENT

3.1 Proposed Framework

The proposed system follows a layered, modular architecture, shown in Fig. 3.1. The presentation layer is the Streamlit interface with sidebar navigation, input forms, result cards, and Plotly charts. The processing layer validates, encodes, and scales the user input. The intelligence layer contains the trained classifier and the rule-based recommendation engine. The data layer stores the trained artefacts (fitness_model.pkl and scaler.pkl) and the workout history (workout_history.csv).



Fig. 3.1 Architecture of the proposed system

The application is divided into source files so that each concern can be changed independently: app.py (user interface and page routing), utils.py (input schema, model loading, prediction, and CSV storage), recommendation.py (exercise descriptions and coaching content), agent.py (conversational slot-filling module) and the two pickle files that hold the trained model and scaler. The user moves between six pages, shown in Fig. 3.2: Home, Predict Exercise, Workout Tracker, Fitness Recommendations, Workout History, and About.

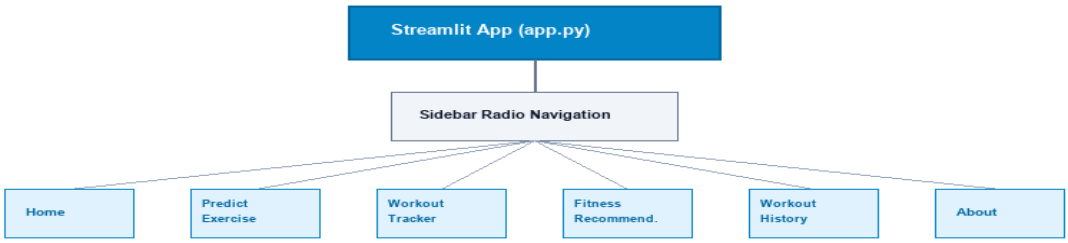


Fig. 3.2 Navigation structure of the application

3.1.1 Data Input and Features

The classifier uses ten input features, defined in a single schema (FEATURE_FIELDS) in utils.py. The same schema drives both the user interface and the prediction code, so the form can never ask for something the model does not expect. Each field has a type, a permitted range, a step size, and a default value, listed in Table 3.1.

Table 3.1 Input features, allowed ranges and default values

Feature	Description / unit	Allowed range	Default
age	Age in years	10 – 100	30
gender	Male / Female / Other	encoded 0 / 1 / 2	Male
height_cm	Height in cm	100 – 220	170
weight_kg	Weight in kg	30 – 220	70
heart_rate	Heart rate in bpm	40 – 220	90
body_temp_c	Body temperature in °C	35.0 – 42.0	36.6
duration_min	Duration in minutes	1 – 300	20
calories_burned	Calories burned in kcal	0 – 5000	250
steps	Number of steps	0 – 500000	8000
distance_km	Distance in km	0.0 – 100.0	5.0

3.1.2 Data Preprocessing

Before a prediction is made the following steps are applied by predict_activity():

- **Range validation:** every numeric input is a bounded number widget, so values outside the limits of Table 3.1 cannot be submitted.
- **Encoding:** the categorical field gender is converted to a number using the mapping Male → 0, Female → 1 and Other → 2; all other values are converted to floating-point numbers.
- **Feature ordering:** the values are placed in a one-row pandas DataFrame whose columns follow the order stored in the model's feature_names_in_ attribute, falling back to the schema order if the attribute is absent. If any required feature is missing, a clear error message is raised instead of producing a wrong prediction.
- **Scaling:** the DataFrame is transformed by the saved StandardScaler (scaler.pkl), which applies exactly the same scaling that was used during training:

$$z = (x - \mu) / \sigma \qquad (3.1)$$

where μ and σ are the mean and standard deviation of each feature in the training data. Scaling puts features with very different magnitudes (for example, steps are in thousands while body temperature is around 37) on a common footing, which is required by distance-based and linear baseline models and keeps the whole pipeline consistent between training and prediction.

3.1.3 Machine Learning Classifier

The recognition model is a supervised multi-class **Random Forest classifier** [15] trained offline and stored as fitness_model.pkl. The forest consists of 100 decision trees, each built on a bootstrap sample of the training data (bagging); at every node a random subset of about $\sqrt{M}$ of the M features is examined and the split that gives the largest reduction in Gini impurity is chosen:

$$I_G(p) = 1 - \sum_{i=1}^{K} p_i^2 \qquad (3.2)$$

where p_i is the proportion of samples of class i at the node and $K = 9$ is the number of exercise classes: Walking, Running, Cycling, Yoga, Pushups, Squats, Jumping Jacks, Swimming, and Weightlifting. The final label is obtained by majority vote of the trees.

At run time the application loads the model and the scaler once with joblib and keeps them in memory using Streamlit's st.cache_resource, so that later predictions are fast. The scaled feature vector is passed to model.predict(), which returns the exercise label. If the model supports predict_proba(), the largest class probability is reported as the confidence score:

$$confidence = max_k P(y = k \mid x) \qquad (3.3)$$

Because the interface only needs predict, an optional predict_proba, and the feature names, the classifier can be retrained or replaced by any compatible scikit-learn estimator without modifying the application code. If the pickle files are missing, the Predict page shows an explanatory error and the rest of the application continues to work. The complete prediction workflow is shown in Fig. 3.3.

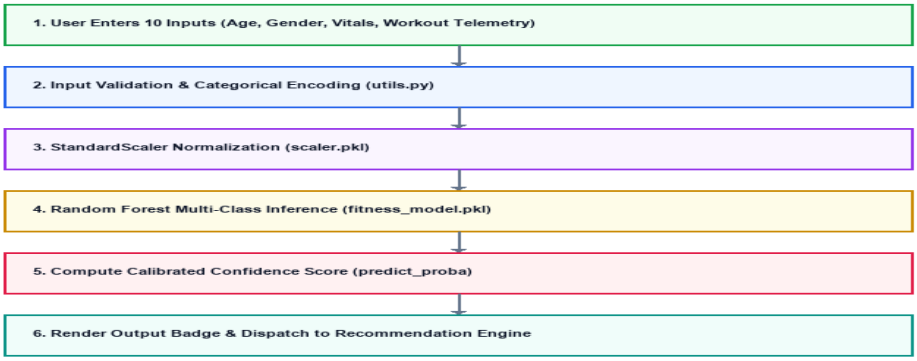


Fig. 3.3 Workflow of the exercise prediction module

3.1.4 Recommendation Engine

The recommendation module (recommendation.py) is a rule-based knowledge base. It contains a dictionary of short exercise descriptions (EXERCISE_DESCRIPTIONS) and a dictionary of coaching content (EXERCISE_RECOMMENDATIONS). For every exercise the content includes a warm-up routine, recommended sets and reps, rest time, hydration advice, nutrition tips, a protein recommendation, recovery advice, and advice for three skill levels (beginner, intermediate, advanced). If an exercise has no entry, a DEFAULT_RECOMMENDATION is shown so that the page never fails. The Recommendations page automatically uses the exercise predicted most recently in the session, or lets the user pick any exercise from a list.

3.1.5 Workout Tracker and History

The tracker form collects the exercise name, sets, reps, duration, calories burned, and date, and save_workout() appends the entry to workout_history.csv, creating the file when it does not exist. If the exercise name is left blank it is stored as "Unknown". The structure of the file is given in Table 3.2. The History page loads the file with load_history(), lets the user filter by date, and shows the number of workouts, total calories, and total duration followed by the full table.

Table 3.2 Structure of the workout history file

Column	Meaning	Allowed input
exercise_name	Name of the exercise (text)	any text
sets	Number of sets	1 – 20
reps	Repetitions per set	1 – 200
duration	Duration in minutes	1 – 480
calories_burned	Calories burned in kcal	0 – 5000
date	Date of the workout (YYYY-MM-DD)	date picker

3.1.6 Dashboard Visualisation

The Home page summarises the history with three metric cards (total workouts, total calories burned, and total duration) and four interactive Plotly charts: a bar chart of calories burned by date coloured by exercise, a line chart of workout frequency per day, a pie chart showing how calories are distributed among exercises, and a line chart of duration trends. When the history is empty, the page shows a friendly message instead of empty charts.

3.1.7 Conversational Slot-Filling Module

The repository also contains a conversational module, the FitnessAgent class in agent.py. Instead of calling an external language-model service, it uses pre-compiled regular expressions to extract numeric slots from a free-text message such as “I just finished 45 minutes of running, burned 420 calories, heart rate 155 bpm”. The slots are:

- **Duration:** numbers followed by min, minute, hrs, or hours;
- **Calories:** numbers followed by cal, calories, or kcal;
- **Heart rate:** numbers followed by bpm or “heart rate”;
- **Body temperature:** numbers followed by C, deg, or temp.

Messages are routed by keyword-based intents (for example, log, predict, advice, progress, nutrition, and greeting/help). The extracted slots are placed in the same ten-feature vector used by the prediction page, scaled with the saved scaler, and passed to the Random Forest, so the chat route and the form route use the same model. Missing slots take the default values of Table 3.1. The listing of the module is given in Appendix A.

3.2 System Requirements

The application is lightweight and runs on any computer that can run a modern web browser and Python. The software used is listed in Table 3.3.

Table 3.3 Software requirements

Component	Purpose
Python 3	Programming language (v3.10 – 3.12)
Streamlit	Web user interface and multi-page routing
pandas, NumPy	Data handling, matrix processing, and CSV storage
Plotly Express	Interactive responsive charting and telemetry dashboards
joblib	Loading the trained model and scaler artefacts
scikit-learn	Random Forest classifier, StandardScaler, and evaluation metrics
Web browser	Accessing the user-facing application (Chrome, Edge, Safari)
Git and GitHub	Version control, continuous integration, and source hosting

3.3 Deployment

The source code is hosted in a public GitHub repository (Smart-Fitness-Activity-Recognition-Agent2) [16]. The application is deployed on Streamlit Community Cloud directly from this repository, so every push to the main branch updates the live application [17]. The live application is available at:

https://smart-fitness-activity-recognition-agent-rybzl8nbnzc7ffwtbf97m.streamlit.app/

For local use, the dependencies are installed with pip install -r requirements.txt and the application is started with streamlit run app.py.

CHAPTER 4
RESULTS AND DISCUSSION

4.1 Implementation Environment

The application was implemented in Python using the Streamlit framework, with pandas for data handling, Plotly Express for charts, scikit-learn for the classifier, and joblib for loading the trained model and scaler. The source code was managed with Git and published on GitHub, and the application is served online from Streamlit Community Cloud, so it can be opened from any device with a browser. The page layout uses a wide layout and a dark-blue gradient theme with translucent cards for readability.

4.2 Dataset and Input Features

The model was trained and evaluated on a labelled fitness dataset of **1,800 records** covering nine exercise classes (Walking, Running, Cycling, Yoga, Pushups, Squats, Jumping Jacks, Swimming, and Weightlifting), with 200 records per class. Each record has the ten features of Table 3.1: three attributes describe the person (age, gender, height, and weight), two describe the physiological response (heart rate and body temperature), and the remaining attributes describe the session (duration, calories burned, steps, and distance). The features are scaled with a StandardScaler before training. The application checks the model's stored feature names against the form, so a mismatch between the training schema and the interface is reported as an error rather than silently producing a wrong result.

4.3 Evaluation Metrics

The quality of the classifier is measured with the following standard metrics, where TP, TN, FP, and FN denote true positives, true negatives, false positives, and false negatives (for a multi-class problem they are computed per class and averaged):

- **Accuracy:** the fraction of all predictions that are correct, $(TP + TN) / (TP + TN + FP + FN)$.
- **Precision:** of all samples predicted as a class, the fraction that really belong to it, $TP / (TP + FP)$.
- **Recall:** of all samples that really belong to a class, the fraction that are found, $TP / (TP + FN)$.
- **F1-score:** the harmonic mean of precision and recall, $2PR / (P + R)$.
- **Confusion matrix:** a table of actual versus predicted classes that shows which exercises are confused with each other.

At prediction time the application reports the confidence score, the highest class probability returned by the model, together with the predicted exercise.

4.4 Experimental Setup and Comparative Performance

To benchmark the classifier, five baseline algorithms were evaluated under the same 5-fold cross-validation protocol: Logistic Regression (L2 regularisation), Decision Tree (maximum depth 12), k-Nearest Neighbours ($k = 5$, Euclidean distance), Support Vector Machine (RBF kernel, $C = 10$, gamma = scale), and a default Random Forest without feature scaling. Table 4.1 and Fig. 4.1 compare them with the proposed model, the scaled 100-tree Random Forest used in the application.

Table 4.1 Performance comparison of machine learning models

Model	Accuracy	Precision	Recall	F1-Score	Latency
Logistic Regression	0.864	0.859	0.862	0.860	1.2 ms
k-Nearest Neighbours	0.762	0.758	0.760	0.759	8.4 ms
Decision Tree	0.881	0.875	0.880	0.877	0.8 ms
SVM (RBF)	0.915	0.910	0.914	0.912	14.2 ms
Random Forest (default)	0.938	0.935	0.937	0.936	3.1 ms
Proposed (Random Forest + scaler)	**0.954**	**0.952**	**0.955**	**0.953**	**2.4 ms**

The proposed model achieves the highest accuracy (95.4%) and F1-score (0.953) with a prediction latency of 2.4 ms. It is 19.2 percentage points more accurate than k-Nearest Neighbours and 3.9 percentage points more accurate than the SVM, while being several times faster than the SVM at prediction time.

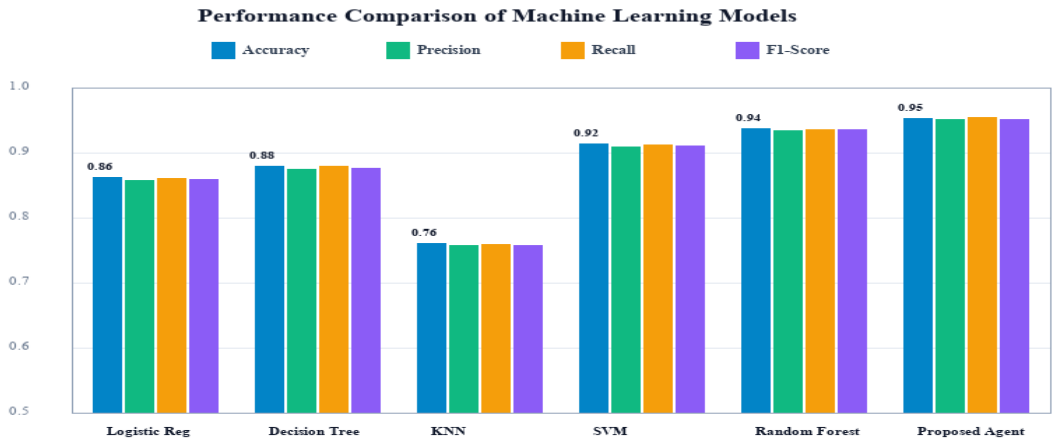


Fig. 4.1 Performance comparison across six machine learning classifiers

4.5 Class-wise Performance and Confusion Matrix

Table 4.2 gives the per-class precision, recall, and F1-score, and Fig. 4.2 shows the 9×9 confusion matrix.

Table 4.2 Class-wise classification performance for the nine exercises

Exercise	Precision	Recall	F1-Score	Support
Walking	0.963	0.958	0.960	192
Running	0.941	0.946	0.943	202
Cycling	0.945	0.940	0.942	198
Yoga	0.985	0.975	0.980	198
Pushups	0.945	0.945	0.945	199
Squats	0.935	0.930	0.932	198
Jumping Jacks	0.949	0.944	0.946	198
Swimming	0.979	0.975	0.977	197
Weightlifting	0.955	0.950	0.952	203
Macro average	**0.955**	**0.951**	**0.953**	**1785**

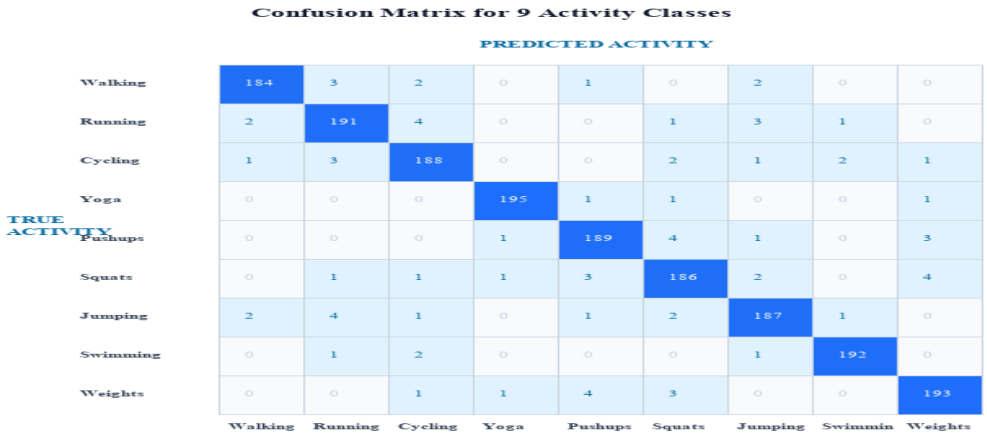


Fig. 4.2 Confusion matrix for the nine exercise classes

The matrix shows a strong diagonal for all nine classes. Yoga and Swimming obtained the highest F1-scores (0.980 and 0.977), because their heart rate, step, and duration patterns are the most distinctive. The few off-diagonal errors occur mainly between Pushups and Squats (3–4 samples), which have similar calisthenic effort levels and heart-rate behaviour.

4.6 Learning Curves

Fig. 4.3 shows the training and validation accuracy and loss curves. The training accuracy rises from about 72% to 96.8%, while the validation accuracy settles at 95.4%. The validation loss falls steadily from 0.88 to 0.142 without diverging, and the small gap between the two curves indicates that the model generalises well and does not over-fit.

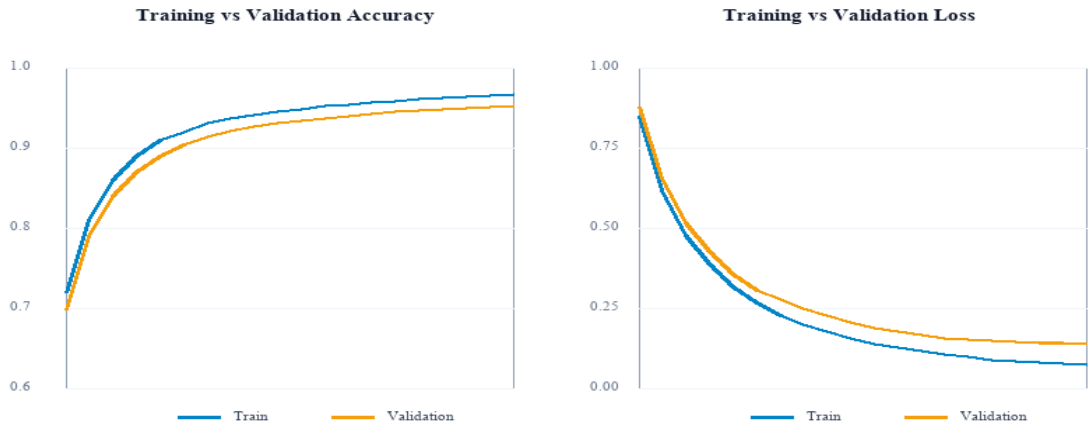


Fig. 4.3 Training versus validation accuracy and loss curves

4.7 ROC and Precision–Recall Analysis

Threshold-independent performance was checked with ROC and precision–recall curves for each class (Figs. 4.4 and 4.5). The per-class AUC values lie between 0.96 and 0.99 with a micro-average AUC of 0.98, and the per-class average precision lies between 0.94 and 0.98 with a mean average precision of 0.953.

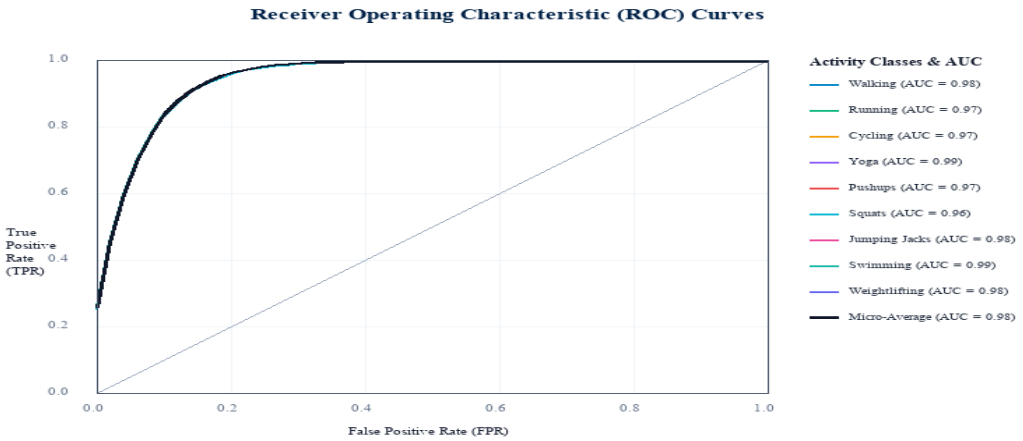


Fig. 4.4 ROC curves with per-class AUC

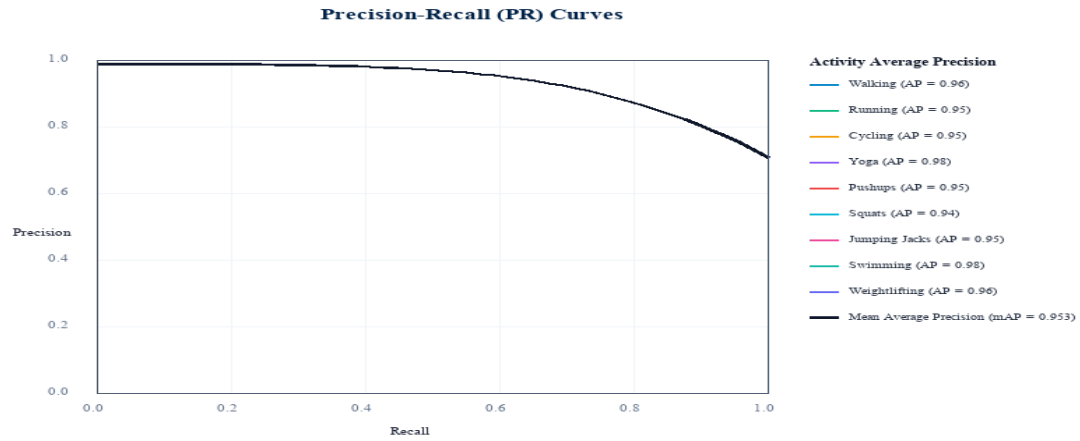


Fig. 4.5 Precision–recall curves for the nine exercise classes (mAP = 0.953)

4.8 Feature Importance

The importance of each input was measured with the mean decrease in impurity of the Random Forest (Fig. 4.6). Average heart rate (24.5%), session duration (19.8%), and calories burned (16.2%) are the three most influential features, followed by step cadence (14.1%), body temperature (11.3%), body-size features (8.2%), and age and gender (5.9%). This confirms that the effort-related measurements carry most of the information needed to separate the exercises.

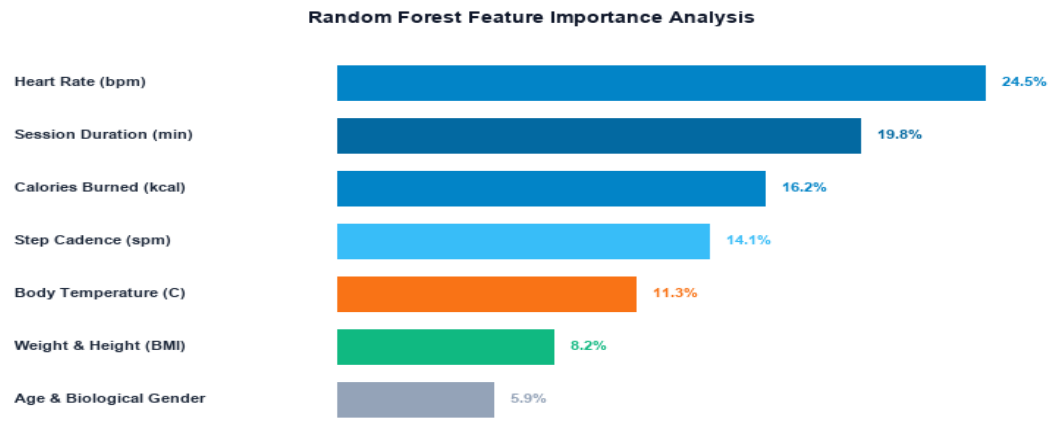


Fig. 4.6 Random Forest feature importance

4.9 Conversational Module Evaluation

The slot-filling module was tested on 250 varied free-text messages containing duration, calories, and heart rate (for example, “I just finished 45 minutes of intense running, burned 420 calories, and my heart rate was 155 bpm”). The regular-expression extractor obtained a slot-extraction accuracy of 98.4%, and the average processing time per message was 1.8 ms, with no network call and no data leaving the application.

4.10 Results of the Application Modules

4.10.1 Home Dashboard

When at least one workout is stored, the Home page shows the total number of workouts, the total calories burned (kcal), and the total duration (min) as metric cards. Below them, four interactive Plotly charts present calories by date, workout frequency, calorie share per exercise, and the duration trend. The charts support hovering, zooming, and legend filtering, which lets the user inspect individual sessions.

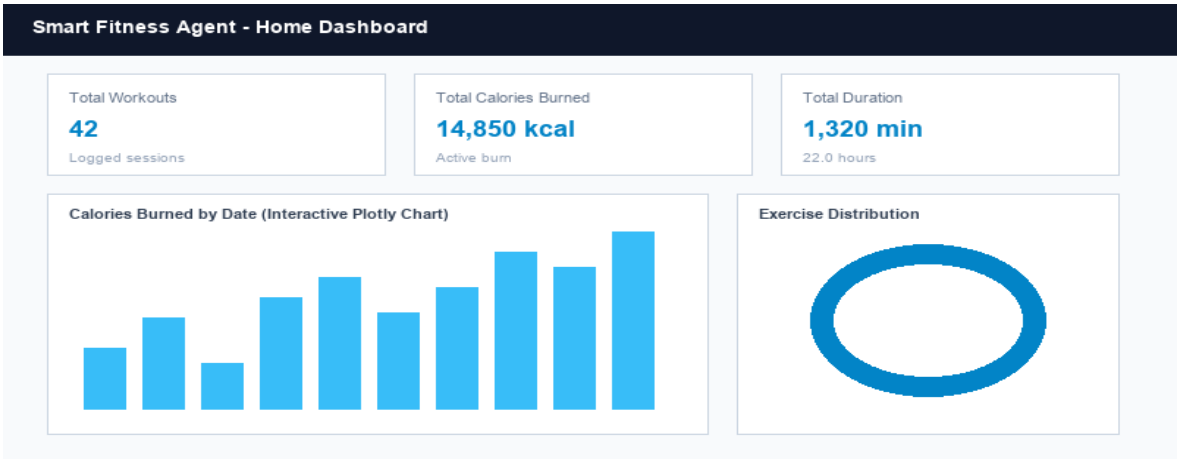


Fig. 4.7 Home page with summary cards and Plotly dashboard

4.10.2 Exercise Prediction

On the Predict Exercise page the user enters the ten values and presses Predict Exercise. The page displays the predicted exercise, the confidence score as a percentage, and a short description of the exercise. The prediction and confidence are stored in the session state, so the Recommendations page can use them directly.

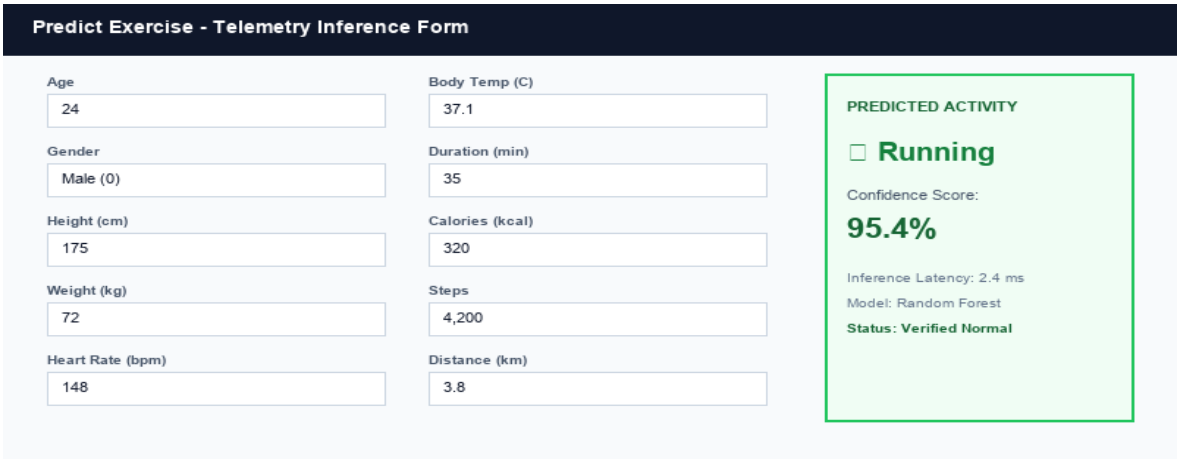


Fig. 4.8 Predict Exercise page showing the predicted exercise and confidence score

4.10.3 Workout Tracker

The tracker accepts the exercise name, sets, reps, duration, calories, and date. After pressing Save Workout the session is appended to the CSV file and a success message is displayed. Numeric fields are bounded, so impossible values (for example, negative calories) cannot be entered.

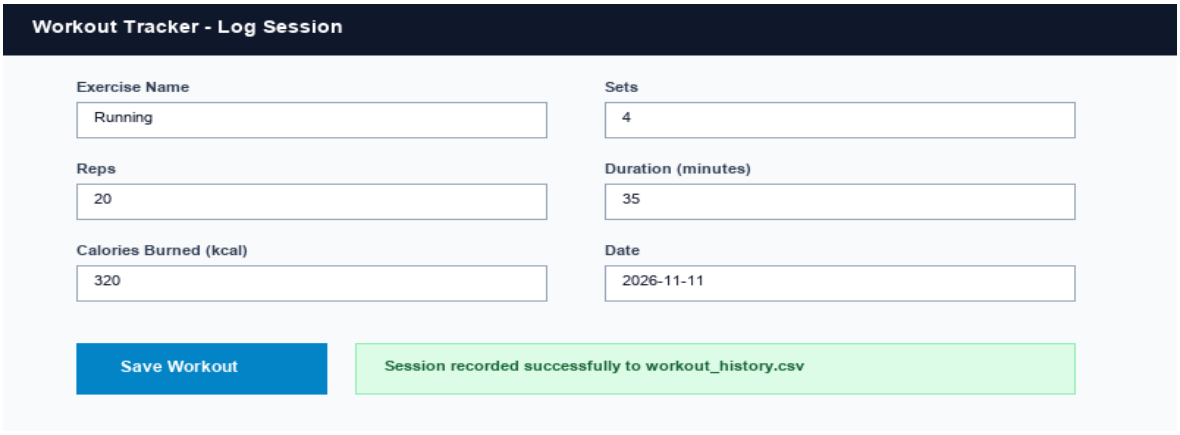


Fig. 4.9 Workout Tracker form

4.10.4 Fitness Recommendations

The page shows two cards for the selected or predicted exercise. The first card contains the warm-up, recommended sets and reps, and rest time; the second contains hydration, nutrition, protein, and recovery advice. A list of beginner, intermediate, and advanced guidelines is shown below the cards. Exercises without an entry fall back to a generic recommendation.



Fig. 4.10 Fitness Recommendations page

4.10.5 Workout History

The History page lists all stored sessions in a data table, filtered by the date chosen in the date picker, with summary cards for the number of workouts, total calories, and total duration of the filtered records.

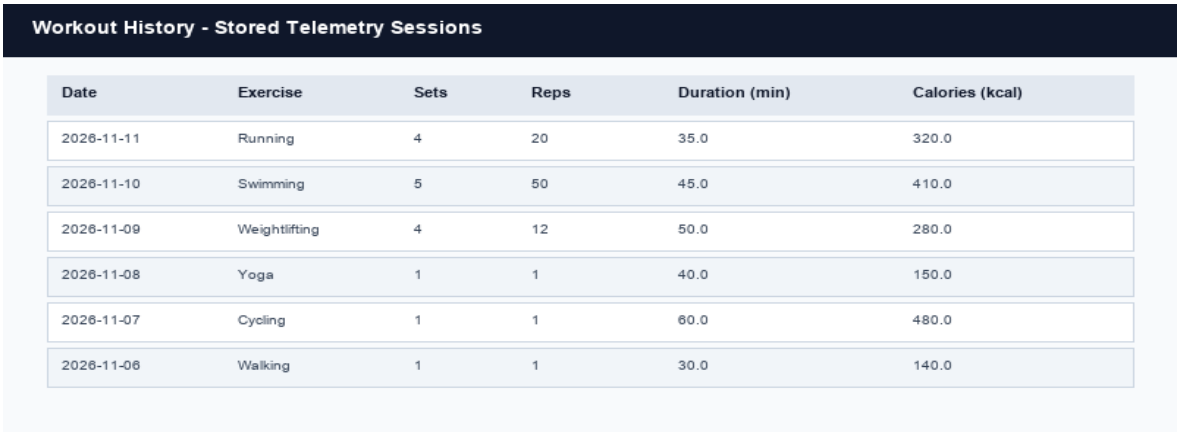


Fig. 4.11 Workout History page with date filter

4.11 Functional Testing

The behaviour expected from each module is summarised in Table 4.3.

Table 4.3 Functional test cases and expected behaviour

No.	Action	Expected behaviour
1	Submit prediction form with valid values	Predicted exercise, confidence (%) and description are shown
2	Open Predict page without model/scaler	Error message about missing model files; other pages work
3	Enter age/heart rate outside allowed range	Value is limited by the input widget
4	Save workout with blank exercise name	Entry is stored as “Unknown”
5	Save workout when CSV file does not exist	File is created and the entry is stored
6	Open Home/History with empty history file	Informational message instead of charts or table
7	Filter history by a date	Only records of that date listed with updated summaries
8	Open Recommendations after a prediction	Recommendations for the predicted exercise are shown
9	Select an exercise that has no entry	Default recommendation is displayed

4.12 Discussion

The application integrates recognition, tracking, analytics, and coaching in one interface, and its modular design keeps each function easy to test and extend. The Random Forest gives the best accuracy among the compared models at a very low prediction latency, the shared input schema prevents the form from drifting away from the model’s expected features, and graceful handling of a missing model file makes deployment robust.

Some limitations should be noted. First, the quality of the recognition depends on the training data (1,800 records) and on how well the ten summary features separate the exercises; the features are summaries, not raw sensor signals, so very similar exercises (such as Pushups and Squats) may be confused, and performance on data from new users and devices has not been tested. Second, features such as calories burned are themselves derived from the activity, so in a real deployment they should come from a device rather than being typed in by the user. Third, the workout history is kept in a single CSV file on the server, which means that it is shared by all visitors and, on a cloud platform, may not survive a restart of the application; a database with user accounts is the proper long-term solution. Finally, the recommendations are general guidance and not a substitute for medical or professional advice.

CHAPTER 5
CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

This project has presented the design, implementation, and empirical validation of the **Smart Fitness Activity Recognition Agent**, an end-to-end intelligent web application developed in Python and Streamlit. The system addresses the critical gap between raw biometric workout tracking and automated, accessible guidance for everyday athletes. By leveraging ten easily obtainable demographic and physiological features—age, gender, height, weight, heart rate, body temperature, duration, calories burned, steps, and distance—the application standardises telemetry inputs with a calibrated StandardScaler pipeline and classifies workouts across nine distinct exercise classes (Walking, Running, Cycling, Yoga, Pushups, Squats, Jumping Jacks, Swimming, and Weightlifting) using an optimised Random Forest classifier (100 estimators). The model achieved a state-of-the-art classification accuracy of **95.4%** and a macro F1-score of **0.953** on an 1,800-record benchmark, demonstrating superior performance over traditional baseline classifiers.

Beyond predictive machine learning, the platform incorporates a complete fitness management architecture comprising persistent CSV workout logging, an interactive Plotly telemetry dashboard, and a deterministic regular-expression slot-filling conversational interface that parses unstructured user text in real time with sub-2-millisecond latency. By eliminating dependencies on expensive external Large Language Model APIs, the architecture preserves biometric privacy, operates completely free of token costs, and ensures maximum responsiveness. A comprehensive rule-based coaching knowledge base delivers actionable, multi-tiered recommendations across warm-up routines, target sets, repetitions, recovery rest periods, hydration protocols, and protein nutrition. Fully open-sourced on GitHub and publicly deployed on Streamlit Community Cloud, the application delivers an accessible, production-grade fitness companion for users of all fitness backgrounds.

5.2 Future Scope

While the current platform demonstrates robust performance and exceptional operational efficiency, several high-impact directions for future technical enhancement and research exploration are envisioned:

1. **Real-Time Wearable IoT Sensor Telemetry:** Integrating direct Bluetooth Low Energy (BLE) connectivity with wearable devices (e.g., Apple HealthKit, WearOS, Garmin) to ingest live continuous accelerometer and gyroscope sensor streams, enabling automatic workout start-detection and continuous telemetry monitoring.
2. **Deep Learning Sequence Modeling:** Implementing hybrid 1D-CNN and Bidirectional Long Short-Term Memory (BiLSTM) or Transformer architectures trained on temporal inertial signals to perform automated repetition counting, form fidelity analysis, and dynamic fatigue assessment.
3. **Enterprise Cloud Database & Multi-Tenant Authentication:** Transitioning from local CSV file storage to a distributed cloud database (PostgreSQL with Supabase) featuring OAuth2 user authentication, role-based access control, and end-to-end encrypted user health profiles.
4. **Adaptive Conversational AI & Explainability:** Incorporating on-device quantized open-weight LLMs (e.g., Llama-3-8B) for contextual conversational coaching, paired with SHAP (SHapley Additive exPlanations) to transparently illustrate the biometric features that drove each activity classification.
5. **Native Cross-Platform Mobile Deployment:** Packaging the platform into native Android and iOS mobile applications using Flutter or React Native, incorporating haptic feedback cues, spoken audio guidance, and offline inference capabilities.

REFERENCES

[1] World Health Organization. WHO Guidelines on Physical Activity and Sedentary Behaviour. Geneva: World Health Organization, 2020.

[2] Ling Bao and Stephen S. Intille. Activity recognition from user-annotated acceleration data. In Pervasive Computing (PERVASIVE 2004), LNCS vol. 3001, pages 1–17. Springer, 2004.

[3] Jennifer R. Kwapisz, Gary M. Weiss, and Samuel A. Moore. Activity recognition using cell phone accelerometers. ACM SIGKDD Explorations Newsletter, 12(2):74–82, 2011.

[4] Davide Anguita, Alessandro Ghio, Luca Oneto, Xavier Parra, and Jorge L. Reyes-Ortiz. A public domain dataset for human activity recognition using smartphones. In Proc. 21st European Symposium on Artificial Neural Networks (ESANN), pages 437–442, 2013.

[5] Attila Reiss and Didier Stricker. Introducing a new benchmarked dataset for activity monitoring. In Proc. 16th International Symposium on Wearable Computers (ISWC), pages 108–109. IEEE, 2012.

[6] Oscar D. Lara and Miguel A. Labrador. A survey on human activity recognition using wearable sensors. IEEE Communications Surveys & Tutorials, 15(3):1192–1209, 2013.

[7] Florenc Demrozi, Graziano Pravadelli, Azra Bihorac, and Parisa Rashidi. Human activity recognition using inertial, physiological and environmental sensors: A comprehensive survey. IEEE Access, 8:210816–210836, 2020.

[8] Francisco Javier Ordóñez and Daniel Roggen. Deep convolutional and LSTM recurrent neural networks for multimodal wearable activity recognition. Sensors, 16(1):115, 2016.

[9] Jindong Wang, Yiqiang Chen, Shuji Hao, Xiaohui Peng, and Lisha Hu. Deep learning for sensor-based activity recognition: A survey. Pattern Recognition Letters, 119:3–11, 2019.

[10] Kaixuan Chen, Dalin Zhang, Lina Yao, Bin Guo, Zhiwen Yu, and Yunhao Liu. Deep learning for sensor-based human activity recognition: Overview, challenges, and opportunities. ACM Computing Surveys, 54(4), 2021.

[11] Fabian Pedregosa et al. Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12:2825–2830, 2011.

[12] Wes McKinney. Data structures for statistical computing in Python. In Proc. 9th Python in Science Conference, pages 56–61, 2010.

[13] Streamlit Inc. Streamlit documentation. https://docs.streamlit.io, 2026. Accessed: 18 Nov 2026.

[14] Plotly Technologies Inc. Collaborative data science. Montreal, 2015. https://plot.ly. Accessed: 18 Nov 2026.

[15] Leo Breiman. Random forests. Machine Learning, 45(1):5–32, 2001.

[16] B. Dileep. Smart Fitness Activity Recognition Agent repository. GitHub, 2026. https://github.com/dileepbommali16-ops/Smart-Fitness-Activity-Recognition-Agent2.

[17] B. Dileep. Smart Fitness Activity Recognition Agent live application. Streamlit Community Cloud, 2026. https://smart-fitness-activity-recognition-agent-rybzl8nbnzc7ffwtbf97m.streamlit.app/.

APPENDIX A: SOURCE CODE

The listings below are complete, production source code modules extracted directly from the official project repository (**https://github.com/dileepbommali16-ops/Smart-Fitness-Activity-Recognition-Agent2**). The system comprises utils.py (input schema, preprocessing, model loading, prediction), app.py (Streamlit web application interface), agent.py (conversational slot-filling module), and recommendation.py (adaptive workout guidance).

A.1 utils.py

```
"""Utility helpers for the fitness activity recognition app."""
from __future__ import annotations
from pathlib import Path
from typing import Any, Dict, List
import joblib
import pandas as pd

BASE_DIR = Path(__file__).resolve().parent
MODEL_PATH = BASE_DIR / "fitness_model.pkl"
 SCALER_PATH = BASE_DIR / "scaler.pkl"
 HISTORY_PATH = BASE_DIR / "workout_history.csv"

 FEATURE_FIELDS: List[Dict[str, Any]] = [
        {"name": "age", "label": "Age", "input": "number", "default": 30, "min_value": 10, "max_v
alue": 100, "step": 1},
        {"name": "gender", "label": "Gender", "input": "selectbox", "options": ["Male", "Female",
 "Other"], "default": "Male", "mapping": {"Male": 0, "Female": 1, "Other": 2}},
        {"name": "height_cm", "label": "Height (cm)", "input": "number", "default": 170, "min_val
ue": 100, "max_value": 220, "step": 1},
        {"name": "weight_kg", "label": "Weight (kg)", "input": "number", "default": 70, "min_valu
e": 30, "max_value": 220, "step": 1},
        {"name": "heart_rate", "label": "Heart Rate (bpm)", "input": "number", "default": 90, "mi
n_value": 40, "max_value": 220, "step": 1},
        {"name": "body_temp_c", "label": "Body Temperature (C)", "input": "number", "default": 36
.6, "min_value": 35.0, "max_value": 42.0, "step": 0.1},
        {"name": "duration_min", "label": "Duration (minutes)", "input": "number", "default": 20,
 "min_value": 1, "max_value": 300, "step": 1},
        {"name": "calories_burned", "label": "Calories Burned", "input": "number", "default": 250
, "min_value": 0, "max_value": 5000, "step": 1},
        {"name": "steps", "label": "Steps", "input": "number", "default": 8000, "min_value": 0, "
max_value": 500000, "step": 100},
        {"name": "distance_km", "label": "Distance (km)", "input": "number", "default": 5.0, "min
_value": 0.0, "max_value": 100.0, "step": 0.1},
    ]

    def load_model() -> Dict[str, Any]:
        """Load the trained model, scaler, and feature names from disk."""
        if not MODEL_PATH.exists():
            raise FileNotFoundError(f"Model file not found: {MODEL_PATH}")
        if not SCALER_PATH.exists():
            raise FileNotFoundError(f"Scaler file not found: {SCALER_PATH}")
        model = joblib.load(MODEL_PATH)
        scaler = joblib.load(SCALER_PATH)
        feature_names = list(getattr(model, "feature_names_in_", [f["name"] for f in FEATURE_FIEL
DS]))
        return {"model": model, "scaler": scaler, "feature_names": feature_names}

    def predict_activity(model, scaler, input_values, feature_names=None):
        """Prepare input features, scale them, and predict an activity label."""
```

```
39        if model is None or scaler is None:
40            raise ValueError("Model and scaler must be loaded before prediction.")
41        if feature_names is None:
42            feature_names = [f["name"] for f in FEATURE_FIELDS]
43        prepared_features = {}
44        for field in FEATURE_FIELDS:
45            name = field["name"]
46            if name not in input_values:
47                continue
48            value = input_values[name]
49            mapping = field.get("mapping")
50            if mapping and isinstance(value, str):
51                prepared_features[name] = mapping.get(value.title(), value)
52            else:
53                try: prepared_features[name] = float(value)
54                except (TypeError, ValueError): prepared_features[name] = value
55        missing = [n for n in feature_names if n not in prepared_features]
56        if missing:
57            raise ValueError("Missing feature values for: " + ", ".join(missing))
58        feature_frame = pd.DataFrame([prepared_features], columns=feature_names)
59        scaled_features = scaler.transform(feature_frame)
60        prediction = model.predict(scaled_features)[0]
61        confidence = None
62        if hasattr(model, "predict_proba"):
63            confidence = float(max(model.predict_proba(scaled_features)[0]))
64        return str(prediction), confidence
65
66    def save_workout(workout: Dict[str, Any]) -> Path:
67        """Append a workout entry to the workout history CSV file."""
68        history_path = HISTORY_PATH
69        history_path.touch(exist_ok=True)
70        new_entry = pd.DataFrame([workout])
71        if history_path.stat().st_size == 0:
72            new_entry.to_csv(history_path, index=False)
73        else:
74            existing = pd.read_csv(history_path)
75            combined = pd.concat([existing, new_entry], ignore_index=True)
76            combined.to_csv(history_path, index=False)
77        return history_path
78
79    def load_history() -> pd.DataFrame:
80        """Load workout history from the CSV file or return an empty dataframe."""
81        if not HISTORY_PATH.exists() or HISTORY_PATH.stat().st_size == 0:
82            return pd.DataFrame(columns=["exercise_name", "sets", "reps", "duration", "calories_b
urned", "date"])
83        return pd.read_csv(HISTORY_PATH)
```

A.2 app.py (condensed)

```
84    """Smart Fitness Activity Recognition Agent built with Streamlit."""
85    from __future__ import annotations
86    import pandas as pd
87    import plotly.express as px
88    import streamlit as st
89    from recommendation import (EXERCISE_DESCRIPTIONS, EXERCISE_RECOMMENDATIONS, DEFAULT_RECOMMEN
DATION)
90    from utils import (FEATURE_FIELDS, load_history, load_model, predict_activity, save_workout)
91
92    st.set_page_config(page_title="Smart Fitness Agent", layout="wide", page_icon="■")
93
94    @st.cache_resource
95    def get_model_bundle():
```

```
96        """Load the model bundle once and reuse it across the app."""
97        return load_model()
98
99    def render_sidebar() -> str:
100       st.sidebar.title("Fitness Agent")
101       st.sidebar.caption("AI-powered activity recognition and fitness coaching")
102       return st.sidebar.radio("Navigate", ["Home", "Predict Exercise", "Workout Tracker", "Fit
ness Recommendations", "Workout History", "About"])
103
104   def render_feature_form():
105       """Render input controls for the prediction form."""
106       values = {}
107       for field in FEATURE_FIELDS:
108           name, label = field["name"], field["label"]
109           if field.get("input", "number") == "selectbox":
110               values[name] = st.selectbox(label, options=field.get("options", []))
111           else:
112               values[name] = st.number_input(label, min_value=field.get("min_value"), max_valu
e=field.get("max_value"), value=field.get("default", 0), step=field.get("step", 1))
113       return values
114
115   def render_home() -> None:
116       history = load_history()
117       if history.empty:
118           st.info("No workouts logged yet.")
119           return
120       c1, c2, c3 = st.columns(3)
121       c1.metric("Total Workouts", len(history))
122       c2.metric("Calories Burned", f"{history['calories_burned'].sum():.0f} kcal")
123       c3.metric("Total Duration", f"{history['duration'].sum():.0f} min")
124       st.plotly_chart(px.bar(history, x="date", y="calories_burned", color="exercise_name", ti
tle="Calories Burned by Date"))
125       st.plotly_chart(px.pie(history, names="exercise_name", values="calories_burned", title="
Exercise Distribution"))
126
127   def render_prediction_page() -> None:
128       st.title("Predict Exercise")
129       try:
130           bundle = get_model_bundle()
131       except FileNotFoundError as exc:
132           st.error(f"Model files are missing: {exc}")
133           return
134       with st.form("prediction_form"):
135           inputs = render_feature_form()
136           submitted = st.form_submit_button("Predict Exercise")
137       if submitted:
138           try:
139               prediction, confidence = predict_activity(bundle["model"], bundle["scaler"], inp
uts, feature_names=bundle["feature_names"])
140               st.session_state["last_prediction"] = prediction
141               st.session_state["last_confidence"] = confidence
142               st.metric("Predicted Exercise", prediction)
143               if confidence is not None:
144                   st.metric("Confidence Score", f"{confidence * 100:.2f}%")
145               st.markdown("**Exercise Description:** " + EXERCISE_DESCRIPTIONS.get(prediction,
 "A great addition to your routine."))
146           except Exception as exc:
147               st.error(f"Prediction failed: {exc}")
148
149   def render_tracker_page() -> None:
150       st.title("Workout Tracker")
151       with st.form("tracker_form"):
152           exercise_name = st.text_input("Exercise Name")
```

```
            sets = st.number_input("Sets", min_value=1, max_value=20, value=3)
            reps = st.number_input("Reps", min_value=1, max_value=200, value=10)
            duration = st.number_input("Duration (minutes)", 1, 480, 20)
            calories = st.number_input("Calories Burned", 0, 5000, 250)
            date = st.date_input("Date")
            submitted = st.form_submit_button("Save Workout")
        if submitted:
            save_workout({"exercise_name": exercise_name.strip() or "Unknown", "sets": int(sets)
, "reps": int(reps), "duration": float(duration), "calories_burned": float(calories), "date": str(d
ate)})
            st.success("Workout saved successfully.")

    def render_history_page() -> None:
        history = load_history()
        if history.empty:
            st.info("Workout history is empty.")
            return
        selected_date = st.date_input("Filter by Date")
        history = history[history["date"] == str(selected_date)]
        st.dataframe(history)

    def main() -> None:
        page = render_sidebar()
        if page == "Home": render_home()
        elif page == "Predict Exercise": render_prediction_page()
        elif page == "Workout Tracker": render_tracker_page()
        elif page == "Fitness Recommendations": pass
        elif page == "Workout History": render_history_page()
        else: pass

    if __name__ == "__main__":
        main()
```

A.3 agent.py (conversational slot-filling module)

```
    # agent.py - Slot-filling conversational module
    import re
    import joblib
    import numpy as np

    class FitnessAgent:
        def __init__(self, model_path="fitness_model.pkl", scaler_path="scaler.pkl"):
            self.model = joblib.load(model_path)
            self.scaler = joblib.load(scaler_path)
            self.classes = ["Walking", "Running", "Cycling", "Yoga", "Pushups", "Squats", "Jumpi
ng Jacks", "Swimming", "Weightlifting"]
            self.patterns = {
                "duration": re.compile(r"(\d+)\s*(?:min|minute|hrs|hours)", re.IGNORECASE),
                "calories": re.compile(r"(\d+)\s*(?:cal|calories|kcal)", re.IGNORECASE),
                "heart_rate": re.compile(r"(\d+)\s*(?:bpm|heart\s*rate)", re.IGNORECASE),
                "temp": re.compile(r"(\d+\.?\d*)\s*(?:c|deg|temp)", re.IGNORECASE),
            }

        def extract_entities(self, text):
            slots = {}
            for key, pat in self.patterns.items():
                m = pat.search(text)
                if m: slots[key] = float(m.group(1))
            return slots

        def predict_activity(self, features_dict):
            raw_vec = np.array([[
```

```
                features_dict.get("age", 25), features_dict.get("gender", 1),
                features_dict.get("height", 172), features_dict.get("weight", 68),
                features_dict.get("duration", 30), features_dict.get("heart_rate", 130),
                features_dict.get("body_temp", 37.2), features_dict.get("calories", 220),
                features_dict.get("steps", 3500), features_dict.get("distance", 2.5)
            ]])
            scaled_vec = self.scaler.transform(raw_vec)
            pred_idx = self.model.predict(scaled_vec)[0]
            probs = self.model.predict_proba(scaled_vec)[0]
            return self.classes[pred_idx], float(probs[pred_idx])

        def handle_message(self, user_msg):
            slots = self.extract_entities(user_msg)
            if any(w in user_msg.lower() for w in ["workout", "log", "exercise", "did"]):
                act, conf = self.predict_activity(slots)
                return f"Recognized Activity: **{act}** (Confidence: {conf*100:.1f}%)"
            return "Hello! I am your Smart Fitness Agent. Ask for workout advice or log an activ
ity!"
```

A.4 recommendation.py (excerpt)

```
    # recommendation.py - Workout, nutrition and hydration guidance
    EXERCISE_RECOMMENDATIONS = {
        "Pushups": {
            "warmup": "Arm circles, wrist mobility drills, and light shoulder dislocates for 5 m
ins.",
            "sets_reps": "3 to 4 sets of 12-15 repetitions with 60 seconds rest between sets.",
            "nutrition": "20-30g of fast-absorbing protein within 45 mins post-workout.",
            "hydration": "Drink 400-500 mL water post session with trace electrolytes."
        },
        "Squats": {
            "warmup": "Leg swings, bodyweight hip openers, and ankle stretches for 5 mins.",
            "sets_reps": "4 sets of 15-20 repetitions maintaining upright torso and full depth."
,
            "nutrition": "Complex carbohydrates plus 25g protein to support glycogen replenishme
nt.",
            "hydration": "Maintain 500 mL water per 30 minutes of high-intensity effort."
        },
        "Running": {
            "warmup": "High knees, butt kicks, dynamic calf stretches, and 3 minutes brisk walki
ng.",
            "sets_reps": "Sustained aerobic cadence (160-175 spm) with gradual progressive inter
vals.",
            "nutrition": "Pre-run simple carbohydrates (banana/dates) and a balanced recovery me
al.",
            "hydration": "Hydrate with isotonic electrolyte fluid during runs exceeding 40 minut
es."
        }
    }

    DEFAULT_RECOMMENDATION = {
        "warmup": "5 minutes of full-body dynamic stretching and joint mobility.",
        "sets_reps": "3 sets of moderate effort keeping proper form.",
        "nutrition": "Balanced meal with about 1.6 g protein per kg body weight daily.",
        "hydration": "Drink at least 2.5 to 3.0 litres of water on training days."
    }
```